«Дело» — концепция фокус-хаба для работы с агентами

Oct 8, 2026 · @Nikolay

Главное

«Дело» — личный локальный инструмент, в котором единица внимания — цель. К цели прикрепляются сессии агентов, чаты, ветки, задачи трекера, статьи и заметки. Открыли цель — за 30 секунд поняли, где вы. Запустили агента — он уже знает контекст.

Аналогия — папка дела у следователя. На обложке — где остановились и что дальше. Внутри — ориентировка (бриф цели), материалы (связанные сущности), рапорты оперативников (отчёты агентов) и архив (база знаний). Оперативник получает ориентировку на входе и сдаёт рапорт на выходе.

Три идеи, на которых держится всё:

1. **Цель — единица внимания.** Переключиться на другую работу значит открыть другую цель.
2. **Ориентировка на входе, рапорт на выходе.** Каждая сессия агента начинается с контекста цели и заканчивается записью в неё.
3. **Файлы — истина.** Цель — папка с markdown. Интерфейс, статусы и поиск — производные, их можно пересобрать.

Рабочее название «Дело» выбрано потому, что «заведи дело» звучит лучше, чем «создай агрегат Goal».

Опорные решения

Ваши ответы задают рамку: личный, локальный, пассивный инструмент с агентами разных вендоров и накоплением знаний. Третий столбец — что каждый ответ меняет в концепции.

Вопрос	Ответ	Что из этого следует
Главная боль	Возврат в контекст — для вас; запуск с контекстом — для агентов	Две точки входа в одну цель: обложка для человека, ориентировка для агента

Вопрос	Ответ	Что из этого следует
Аудитория	Только для себя	Нет ролей и прав доступа; можно жёстко подстроиться под свой стек
Масштаб	Много целей и агентов	Нужны карта с сигналами, фильтры и раздел «Неразобранное»
Активность	Пассивная карта	Сигналы видны, когда вы смотрите; единственный тихий сигнал — «агент ждёт вас»
Роль агентов	Исполнители с отчётом	Нужен канал записи обратно в цель — MCP-сервер «Дела»
Структура	Плоские цели	Один тип «Цель», группировка по областям; цель ↔ цель только ссылкой
Трекер	Гибрид	Цель своя; задача Jira — «якорь», её статус только читается
Привязка	Через контекст + вручную	Запущенное из цели привязывается само; остальное — вставкой ссылки
Память	База знаний	Цель копит решения и выжимки; нужны слои, чтобы возврат не утонул в деталях
Интерфейс	Веб/десктоп-дашборд	Локальный сервер + UI в браузере
Хранение	Локальные файлы	Два git-хранилища, рабочее и личное; индекс — одноразовый кэш
Экосистема	Мультивендор	Адаптеры с разной глубиной; общие каналы — файлы и MCP

Конечный сценарий

Инструмент готов, когда работает вот такой день. Всё остальное в документе выведено из него. Цель и детали — условный пример.

09:40 — возврат. Вы открываете карту. Двадцать активных целей сгруппированы по областям. На трёх стоит метка «ждёт вас», на пяти — «новый рапорт». Ничего не мигает и не звенит.

09:41 — обложка. Вы открываете цель «Переезд кэша на Redis». Сверху три строки:

- Остановился: выбор стратегии инвалидации.
- Сделано: бенчмарк готов (рапорт Codex, вчера).
- Дальше: решить про TTL для поиска.

Контекст восстановлен до того, как вы прокрутили страницу.

09:43 — запуск. Вы нажимаете «Новый агент», выбираете Claude Code и пишете одну строку задачи. Инструмент собирает ориентировку, создаёт ветку и запускает агента. Сессия и ветка уже привязаны к цели.

11:30 — рапорт. Агент закончил и записал рапорт: что сделал, что решил, что осталось. Рапорт лёг в журнал, обложка обновилась. Вы увидите это, когда сами заглянете на карту.

Через месяц — архив. Цель закрыта. Её решения и выжимки статей находятся поиском из любой новой цели и попадают в ориентировки следующих агентов.

Принципы проектирования

Семь правил, которые разрешают спорные решения дальше. Если фича нарушает одно из них, она не нужна.

1. **Цель — единица внимания.** Всё остальное — её спутники. Нет экрана «все сессии» без привязки к целям, кроме «Неразобранного».
2. **Файлы — истина, остальное производное.** Индекс, статусы и UI можно удалить и пересобрать из папки целей.
3. **Бриф толкается, знания тянутся.** В ориентировку агента попадает компактный бриф и оглавление знаний. Детали агент запрашивает сам, иначе база знаний съест его контекстное окно.
4. **Карта молчит, пока не спросишь.** Сигналы вычисляются постоянно, но показываются только на карте. Единственное исключение — «агент ждёт вас»: тихий счётчик или беззвучное уведомление. Никаких подсказок «чем заняться».
5. **Симметрия входа и выхода.** Сессия агента = ориентировка → работа → рапорт. Сессия без рапорта помечена как незакрытая.

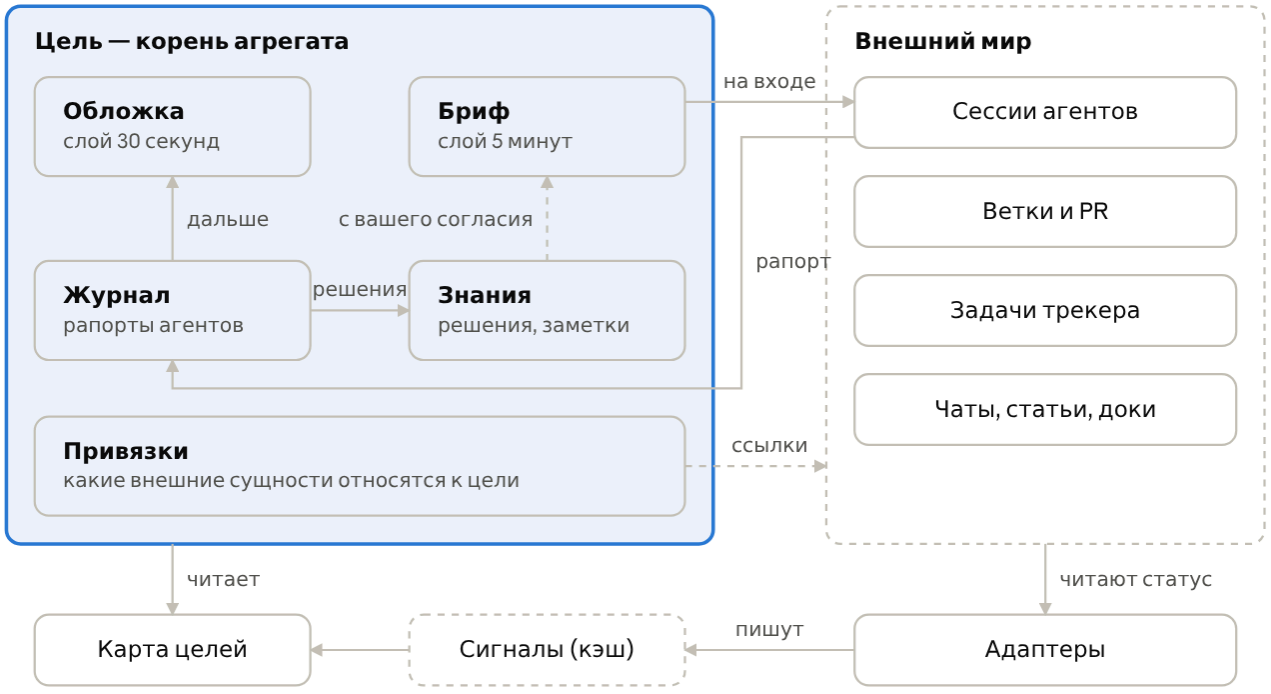
6. **Слои внимания.** 30 секунд — обложка, 5 минут — бриф и журнал, сколько угодно — база знаний. Нижний слой никогда не лезет в верхний.
7. **Вендор подключён настолько, насколько позволяет.** Адаптер честно объявляет свои возможности, а инструмент работает даже с минимальным уровнем — ручной ссылкой.

Доменная модель

Цель — корень агрегата: внутри неё обложка, бриф, привязки, журнал и знания. Внешними сущностями (сессиями, ветками, задачами) цель не владеет — только ссылается. Их статус — отдельная проекция, которую считают адаптеры.

Цель владеет содержимым, внешний мир — только по ссылке

Сплошная стрелка — данные, пунктир — ссылка или производное



доменная модель · агрегат «Цель» и внешний мир

Агент получает бриф на входе и возвращает рапорт в журнал. Из журнала решения уходят в знания, «дальше» — на обложку. Статусы внешнего мира живут отдельно, в кэше сигналов, и цель их не хранит.

Эскиз типов на TypeScript — не схема хранения, а язык предметной области:

```
// Корень агрегата. Всё внутри меняется только через цель.
interface Goal {
  id: GoalId;
  title: string;
```

```
  why: string;                  // зачем
  doneWhen: string;             // критерий готовности
  state: 'active' | 'paused' | 'done' | 'dropped';
  areas: string[];              // группировка, а не иерархия
  anchor?: TrackerRef;          // гибрид: задача трекера как якорь
  cover: Cover;                 // слой «30 секунд»
  brief: Brief;                 // слой «5 минут», основа ориентировки
  links: Link[];                // материалы дела
  journal: Report[];            // рапорты агентов и ваши записи
  knowledge: KnowledgeItem[];
}

interface Cover { stoppedAt: string; done: string; next: string; source:
'report' | 'me' }
interface Brief { context: string; constraints: string[]; keyDecisions:
DecisionId[] }

interface Link {
  id: LinkId;
  kind: 'agent-session' | 'branch' | 'pr' | 'tracker' | 'chat' | 'article' |
'doc';
  ref: string;                  // URL, id сессии, имя ветки
  title: string;
  boundBy: 'context' | 'manual';
  vendor?: string;              // для сессий: claude-code, codex, cursor…
}

interface Report {
  sessionLinkId?: LinkId;       // кто сдал
  at: string;
  did: string[]; decided: string[]; open: string[]; next: string[];
  auto: boolean;                // true — собран из лога, агент не отчитался
}

type KnowledgeItem =
  | { kind: 'decision'; id: DecisionId; title: string; because: string }
  | { kind: 'note'; body: string }
  | { kind: 'source'; url: string; extract: string };

// НЕ часть цели: проекция, которую считают адаптеры. Живёт в кэше.
interface Signal {
  linkId: LinkId;
  state: 'running' | 'waiting-you' | 'finished' | 'failed' | 'unknown';
  seenAt: string;
}
```

События домена — что происходит с целью:

- `GoalOpened` — цель заведена.
- `EntityLinked` — к цели привязана сущность (из контекста или вручную).
- `AgentLaunched` — агент ушёл с ориентировкой.
- `ReportFiled` — рапорт сдан. Реакция: пункты `decided` становятся решениями в знаниях, `next` попадает на обложку.
- `DecisionRecorded` — решение зафиксировано (вами или из рапорта).
- `GoalPaused`, `GoalResumed`, `GoalClosed` — жизненный цикл.

Инварианты:

- Сессия агента принадлежит ровно одной цели.
- Обложка — не длиннее трёх строк.
- Статус внешней сущности никогда не хранится в цели — только `Signal` в кэше с временем снятия.
- Закрытая цель не принимает новых привязок, но её знания остаются в поиске.

Цель изнутри: четыре слоя

База знаний и быстрый возврат тянут в разные стороны. Конфликт снимается слоями: чем глубже слой, тем больше в нём и тем реже вы туда смотрите.

Слой	Время чтения	Что внутри	Кто пишет
Обложка	30 секунд	Три строки: где остановился, что сделано, что дальше	Последний рапорт; ваша строка важнее рапорта
Бриф	5 минут	Зачем, критерий готовности, ограничения, ключевые решения	Вы; агент может предложить правку в рапорте
Журнал	По ситуации	Рапорты агентов и ваши записи, новые сверху	Агенты и вы, только дописывание
Знания	Через поиск	Решения с обоснованием, заметки, выжимки статей	Автоматически из рапортов + вы вручную

Информация движется снизу вверх и сжимается: рапорт ложится в журнал, его решения — в знания, его «дальше» — на обложку. В бриф ничего не попадает без вашего согласия: это то, что читает каждый следующий агент, и ошибка там размножается.

При закрытии цели обложка превращается в итог из одной строки, бриф замораживается, знания остаются в общем поиске.

Агенты: ориентировка и рапорт

Каждая сессия проходит один и тот же цикл, независимо от вендора. Общих каналов два: текст (его читает любой агент) и MCP-сервер «Дела» (его подключает любой современный агент).

1. **Ориентировка.** Инструмент собирает её из цели: бриф, обложка, список привязок, оглавление знаний, задача сессии и инструкция «в конце сдай рапорт». Передаётся как стартовый промпт или файл инструкций в рабочей папке.
2. **Запуск.** Инструмент создаёт ветку (или git worktree) цели, запускает агента в отдельной вкладке herdr и сразу привязывает сессию и ветку к цели.
3. **Работа.** Агент тянет нужные знания через MCP (`search_knowledge`) и может привязать найденное (`link_entity`). Адаптер следит за статусом сессии.
4. **Рапорт.** Агент вызывает `file_report`: что сделал, что решил, что открыто, что дальше. Если сессия закончилась без рапорта, инструмент собирает его из лога сессии и помечает как «авто».

Адаптеры вендоров объявляют, что умеют. Инструмент подстраивается под каждый уровень вместо того, чтобы требовать максимума:

Возможность	Что даёт	Если адаптер не умеет
Запуск	Агент стартует из цели с ориентировкой	Кнопка «Скопировать ориентировку»
Обнаружение	Сессии, запущенные мимо «Дела», попадают в «Неразобранное»	Только ручная привязка
Статус	«Работает / ждёт вас / закончил» на карте	Статус «неизвестно», виден только факт сессии
Возобновление	Клик — и вы в живой сессии	Ссылка или команда для терминала

Возможность	Что даёт	Если адаптер не умеет
Рапорт	Агент пишет в цель через MCP	Авто-рапорт из лога или ваша запись

Первый адаптер — Claude Code: у него есть хуки жизненного цикла сессии и возобновление по id, так что он закрывает все пять строк. Облачные агенты и чаты (Cowork, claude.ai) локально не видны — для них только ручная ссылка и скопированная ориентировка.

Привязка и «Неразобранное»

Сущность попадает в цель одним из двух способов, а то, что не попало никуда, ждёт в одном месте.

Через контекст. Контекст — это «нахожусь внутри цели X». Он возникает тремя путями:

- агент запущен кнопкой из цели;
- в терминале выполнено `delo use <цель>` — всё, что запущено в этом терминале, получает переменную окружения с id цели;
- работа идёт в ветке или worktree, созданных для цели, — PR из этой ветки привяжется сам.

Вручную. Вы вставляете ссылку в цель. Инструмент по URL узнаёт тип (задача трекера, PR, чат, статья) и подтягивает заголовок. Для статьи можно сразу попросить выжимку — она ляжет в знания.

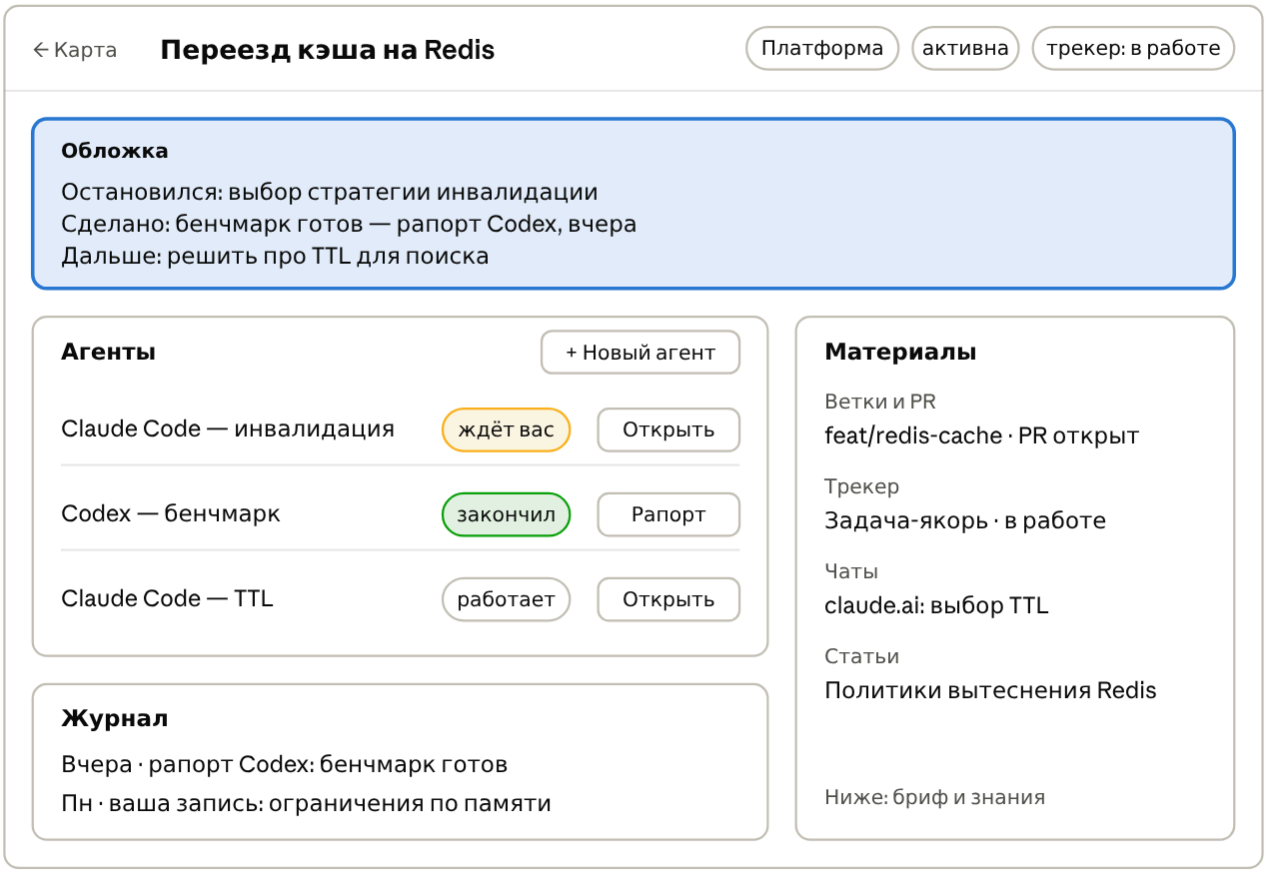
«Неразобранное» — страховка для масштаба «много всего». Адаптеры находят сессии и ветки без цели и складывают их в один список. У каждой находки два действия: «в цель…» и «игнорировать». Это счётчик на карте, а не уведомление.

Распознавание по конвенциям (ключ задачи в имени ветки) вы не выбрали как автопривязку. Его можно добавить позже как подсказку в «Неразобранном»: «похоже на цель X — привязать?».

Интерфейс

Два основных экрана: карта всех целей и страница одной цели. Всё остальное — диалоги поверх них.

Страница цели читается сверху вниз: обложка, агенты, материалы



макет страницы цели · условный пример

Обложка отвечает на «где я» до прокрутки. Один агент ждёт ответа — это видно по метке, а не по уведомлению.

Карта целей — точка входа.

- Цели сгруппированы по областям, фильтр по состоянию (активна / на паузе / закрыта).
- Карточка цели: название, строка «Дальше: …» с обложки, метки «ждёт вас», «работает», «новый рапорт», время последней активности.
- Внутри области сначала цели с «ждёт вас», потом по свежести. Сортировка — не подсказка: она ничего не предлагает, только упорядочивает.
- Сверху — счётчик «Неразобранное» и поиск по знаниям всех целей.

Страница цели — сверху вниз, от быстрого к глубокому:

1. Обложка из трёх строк и якорь Jira со статусом.
2. Агенты: сессии со статусами и кнопками «Открыть», «Возобновить», «Новый агент».
3. Материалы: ветки и PR, задачи трекера, чаты, статьи — сгруппированы по типу, со статусами.

4. Журнал: рапорты, новые сверху; авто-рапорты помечены.
5. Бриф (редактируется на месте) и знания цели.

Переход в сессию агента. Кнопка «Открыть» — это ссылка вида delo://session/<id>. Система через xdg-open передаёт её обработчику, а тот фокусирует окно foot и нужную вкладку herdr через протокол фокуса herdr. Для облачных агентов и чатов — просто ссылка.

Хранение и архитектура

Всё живёт на вашей машине: два хранилища целей в git (рабочее и личное), локальный демон на Node.js и TypeScript, веб-интерфейс на localhost. Один язык от типов домена до UI.

Раскладка файлов. Агент или вы можете прочитать любую цель без инструмента — это просто markdown.

```
~/delo/
  work/                           # рабочее хранилище: свой git,
синхронизация только с корпоративным git или никуда
    goals/
      redis-cache/
        goal.md                   # frontmatter: id, состояние, области,
якорь; тело: обложка + бриф
        links.yaml                # привязанные сущности
        journal/
          2026-10-08-claude-a1b2.md   # рапорт
        knowledge/
          decisions/  notes/  sources/
  personal/                       # личное хранилище: отдельный git,
синхронизация по вашему выбору
    goals/ …
  .cache/                         # индекс SQLite и сигналы; вне git,
пересобирается
```

Компоненты демона:

- **Хранилище** — читает и пишет папки целей, следит за изменениями на диске (вы можете править файлы в редакторе).
- **Индекс** — SQLite с полнотекстовым поиском по знаниям; семантический поиск — позже. Удалили — пересобрался.
- **Адаптеры** — агенты (Claude Code, Codex, Cursor…) и источники (git/GitHub, трекер). Пишут только в кэш сигналов и в «Неразобранное»; событие «агент ждёт вас» дополнительно уходит в тихий сигнал на рабочем столе.

- **Запускатель** — собирает ориентировку, создаёт ветку, поднимает агента во вкладке herdr.
- **MCP-сервер** — интерфейс для агентов: `get_brief`, `search_knowledge`, `link_entity`, `file_report`.
- **HTTP API** — для веб-интерфейса; плюс обработчик ссылок delo://, который фокусирует окно и вкладку агента.

Десктопная обёртка (например, Tauri) — только если вкладка браузера начнёт мешать. Архитектуру она не меняет.

Путь к MVP

Идём от конечного сценария назад. Обложке нужны рапорты, рапортам — запуск с ориентировкой, ориентировке — цель в файлах. Поэтому первый этап проверяет цикл «ориентировка → рапорт», а карта и знания идут после.

Ближайший шаг, 15 минут, без кода: заведите руками папку одной реальной цели с `goal.md` (обложка + бриф). Вставьте бриф в промпт агента и попросите в конце написать рапорт в `journal/`. Готово, когда в папке лежит первый рапорт и вы можете сказать, помог ли он вернуться в контекст.

1. **Этап 1. «Папка дела» — цикл для одного вендора.**
   - Цели как папки; CLI `delo new` и `delo launch <цель> claude`.
   - Ориентировка, ветка, вкладка herdr, автопривязка сессии.
   - MCP с `file_report`; обложка из последнего рапорта.
   - Простая страница цели только для чтения и переход в сессию по ссылке delo://.
   - Готово, когда неделю ведёте через «Дело» 3–5 реальных целей и возврат в цель занимает меньше минуты.
2. **Этап 2. «Карта» — много целей и вендоров.**
   - Карта целей с сигналами, «Неразобранное» и тихий сигнал «агент ждёт вас».
   - Ручная привязка по URL, якорь Jira со статусом, автоитог из лога для сессий без рапорта.
   - Второй адаптер агента — проверка, что абстракция адаптера верная.
   - Готово, когда утро начинается с карты, а не с обхода вкладок и терминалов.
3. **Этап 3. «Архив» — знания и глубина.**
   - Поиск по знаниям всех целей и `search_knowledge` для агентов.

- Выжимки статей по ссылке.
- Готово, когда решение из закрытой цели само находится агентом в новой.

Риски и принятые решения

Главный риск — дисциплина рапортов: без них обложка пустеет, и вся концепция становится каталогом ссылок. По каждому риску и открытому вопросу решение принято 8 октября 2026 года.

Риск	Решение
Сессия закончилась без итога — обложка устаревает	Автоитог из лога сессии с пометкой «автоматически»
Агент ждёт ответа, а карту никто не открывает	Единственное исключение из пассивности: тихий сигнал «агент ждёт вас»
Обложка расходится с вашим пониманием	Агент обновляет её из итога; ваша правка важнее
Знания раздувают стартовый контекст агента	На старте — только описание цели и оглавление знаний; детали агент ищет сам
Рабочие данные утекут при синхронизации	Два хранилища; рабочее синхронизируется только с корпоративным git или никуда
Большой проект распухает в одну цель	Резать на цели одной области со ссылками; иерархию и блокировки не вводим
Переход в CLI-сессию из браузера	Закрыт: ссылка delo:// → xdg-open → фокус окна foot и вкладки herdr
Облачные агенты не видны локально	Риск низкий: работа в основном локальная; облачное — ручной ссылкой
Нет доступа с телефона	Личное хранилище читается markdown-клиентом; рабочее — только если позволяет политика компании

Трекер:

первым якорем рабочих целей будет Jira. Адаптер второго этапа только читает статус задачи по её ключу и ничего не пишет в Jira. Инструмент работает локально, поэтому корпоративная Jira за VPN проблемой не будет.